



深圳技术大学

SHENZHEN TECHNOLOGY UNIVERSITY

本科毕业论文（设计）

题目： 上机考试信息安全收发 CS 软件体系设计

姓 名： 范佳铭

学 院： 大数据与互联网学院

专 业： 计算机科学与技术

学 号： 202100202085

指导教师： 郭云镛

职 称： 助理教授

提交日期： 2025 年 5 月 9 日

深圳技术大学本科毕业论文（设计）诚信声明

本人郑重声明：所呈交的毕业论文（设计），题目《上机考试信息安全收发 CS 软件体系设计》是本人在指导教师的指导下，独立进行研究工作所取得的成果。对本文的研究做出重要贡献的个人和集体，均已在文中以明确方式注明。除此之外，本论文不包含任何其他个人或集体已经发表或撰写过的作品成果。本人完全意识到本声明的法律结果。

毕业论文（设计）作者签名：

日期： 年 月 日

目 录

摘要.....	I
Abstract.....	II
1 引言	1
1.1 课题的研究背景与意义.....	1
1.2 国内外发展状况.....	2
1.2.1 技术应用现状	2
1.2.2 基于 C/S 架构与对称加密的图书馆管理系统	4
1.2.3 基于 C/S 模式的远程网络视频监控系统.....	5
1.3 论文组织结构	6
2 关键技术概述	7
2.1 C/S 软件体系.....	7
2.2 Electron 技术简介	8
2.3 Node-Media-Server 流媒体服务器.....	9
2.4 FFmpeg 音视频处理工具	10
2.5 WebSocket 实时通信技术.....	12
2.5.1 WebSocket 协议原理	12
2.5.2 技术优势与特点	13
2.5.3 在监考系统中的应用	14
2.6 本章小结.....	14
3 需求分析.....	15
3.1 涉考人员相关需求分析.....	15

3.2	功能需求分析	15
3.3	服务端需求分析	16
4	系统设计与实现	17
4.1	客户端设计与实现	17
4.1.1	主进程与应用管理模块	17
4.1.2	媒体服务模块	19
4.1.3	API 服务与通信模块	19
4.1.4	配置管理模块	20
4.2	服务端设计与实现	21
4.3	线下考试无纸化密码分发	22
5	系统测试	24
5.1	测试环境配置	24
5.2	功能测试	25
5.3	性能测试	26
5.4	基础安全检查	27
5.5	测试结论与展望	27
6	总结与展望	28
	参考文献	28
	致谢	31

上机考试信息安全收发 CS 软件体系设计

【摘 要】 随着在线教育和远程考试的普及，如何保障考试的公平性与安全性成为重要挑战。传统监考方式难以适应线上场景，现有远程监控工具亦存在不足。针对这些问题，本设计旨在开发一个基于 C/S 架构的安全考试信息收发与监控系统。系统客户端采用 Electron 框架构建，负责实时捕获考生屏幕与摄像头画面、监控系统进程与资源占用，并通过 FFmpeg 和 Node-Media-Server 进行视频流处理与本地分发。服务端基于 Golang 和 Gin 框架开发，负责管理客户端连接、分发视频流地址、通过 WebSocket 实现与客户端的实时双向通信（如下发指令、接收状态信息）。系统重点关注安全性，采用加密存储本地配置、安全分发服务器访问凭证、TLS/SSL 加密通信、Token 认证等机制，并设计了禁止随意退出、进程黑名单等防作弊措施。通过在 Windows 和 Ubuntu 环境下进行全面的性能、兼容性、安全性及弱网测试，结果表明该系统功能完备、性能稳定、兼容性良好、安全性高，能够有效满足在线考试或竞赛的实时监控需求，提升监考效率与考试公平性。

【关键词】 在线考试；远程监考；C/S 架构；Electron；流媒体；信息安全

Design of C/S Software Architecture for Secure Information Transmission in Computer-based Examinations

【Abstract】 With the popularization of online education and remote examinations, ensuring fairness and security has become a significant challenge. Traditional proctoring methods struggle to adapt to online scenarios, and existing remote monitoring tools have limitations. To address these issues, this project aims to develop a secure examination information transmission and monitoring system based on the C/S architecture. The client-side, built with the Electron framework, is responsible for real-time capture of the examinee's screen and camera feed, monitoring system processes and resource usage, and processing/distributing the video stream locally using FFmpeg and Node-Media-Server. The server-side, developed with Golang and the Gin framework, manages client connections, distributes video stream addresses, and facilitates real-time bidirectional communication with clients via WebSocket (e.g., sending commands, receiving status information). The system emphasizes security, employing mechanisms such as encrypted local configuration storage, secure distribution of server credentials, TLS/SSL encrypted communication, Token authentication, and anti-cheating measures like preventing unauthorized termination and process blacklisting. Comprehensive functional, performance, compatibility, security, and weak network tests conducted in Windows and Ubuntu environments demonstrate that the system is functionally complete, performs stably, exhibits good compatibility, and offers high security. It effectively meets the real-time monitoring requirements for online examinations or competitions, enhancing proctoring efficiency and examination fairness.

【Key words】 Online Examination; Remote Proctoring; C/S Architecture; Electron; Streaming Media; Information Security

1 引言

1.1 课题的研究背景与意义

考试的公平与安全两个方面一直都是备受考验的，早期传统化的监考方式主要依赖于监考人员的现场监控，这种监考方式在一些重要纸质考试仍具备实际意义，然而在大型线上线下的赛事上存在着许多问题，如监控效率低下等，尤其是在线上赛事上明显会出现监控成本高的问题。

疫情期间，线上上机考试所具备的高效、低成本、免接触等特性，被越来越多的高校以及企业机构所青睐，线上考试无需准备纸质试卷，考后统一批卷，同时，也不需要参考者不远千里前来参加线下考试，这些种种，无不凸显了远程考试、监考、批卷的便利性。因此，越来越多企业开始采用线上笔试、面试来进行招聘工作，然而，如何保障考试的公平和安全又成为了亟需解决的问题。

而安全性问题尤其突出。考试前的密码分发如果被截获或篡改，将直接威胁到考试的公平性，因此也诞生了一些基于机器学习的入侵检测模型，用于检测 SSH 协议上的用户名枚举攻击^[1]。而在考试中，如果未能及时有效地检测并防止违规行为，将极大降低考试的权威性和可信度。因此，构建一个安全、可靠、实时的考试信息收发系统，对实现考试信息化、提升管理效率具有重要意义。

我国教育部曾提出，要加强教育信息化建设，促进信息技术与教育的深度融合。而随着新冠疫情的爆发，“停课不停学”政策的推行，进一步加速了在线教育与考试的普及，也对信息安全提出了更高要求。

基于以上背景以及需求，本选题旨在设计一个基于 C/S 架构的安全考试信息收发系统，该系统主要包含以下特性：

监考服务器访问密码安全分发与存储。考生无法得知监考服务器的访问地址以及密码，也无法直接通过查看本地配置文件获得相关信息。

考中信息实时监控与传输。考试期间，可以采用电脑屏幕监控、进程黑名单监控、人脸识别等技术，来提升考试的安全性以及公平性。在数据传输方面，可以适时地结合并行化 Rsync 工具，来进行数据相关操作。^{[2][3]}

考试数据的无纸化统计。通过适时地采集本地考试机数据，来辅助监考人员及时获取考生的相关信息，如 CPU 占用率、内存使用情况、网络流量等，这些信息有助于辅助监考人员判定考生考试合法性。

通过实现上述功能，既能满足教育场景对安全性、实时性和可靠性的需求，也为后续电子考试系统的推广和升级提供实践依据。

这样看下来，传统的现场监考方式在远程环境下难以进行，而现有的远程监控工具往往存在配置复杂、易被绕过、数据安全性不足等问题，难以满足高标准的考试监控需求。因此，本毕设旨在开发一款专为线上编程竞赛或是常规线上考试场景设计的直播监控工具，通过研究市面上现有的方案，以及对实际情况的考虑，本工具基本实现了对考场环境的实时、全面监控。

该工具通过在考试前由监考志愿者配置服务器地址和访问密码，这种配置方式提高了工具在不同环境下的泛用性，此外，所有本地配置信息均采用加密存储，防止考生通过本地手段篡改或破解，确保监控环境的安全性和可靠性。

监控工具系统启动后，客户端会在本地自动维护本地直播流，实时采集并上传考生的屏幕和摄像头画面，便于监考人员远程监督。同时，客户端还可根据服务器要求，定时收集并上报本地机器的关键信息（如进程列表、CPU 占用、MAC 地址、IP 地址等），进一步提升监控的全面性和防作弊能力。

通过这些设计方案，本系统不仅提升了远程考试、线上考试的安全性和管理效率，也为后续相关领域的研究和应用提供了技术基础，有助于推动智能化、自动化考试监控技术的发展，促进教育公平与公正。

1.2 国内外发展状况

1.2.1 技术应用现状

为了更好地满足当今对于在线考试的需求，国内外也出现了很多的在线监考平台。这些在线考试系统主要通过提供线上登录、发卷、判题、批卷、作弊检测等等功能，来保证线上考试的正常运行。

在国内，考试信息化系统的发展已初具规模。以部分高校为例，已有基于 C/S 架构的考试系统实现了考试管理、监控与成绩分析等功能。然而，许多系统仍在

安全性和实时性方面存在不足：

考试信息分发主要依赖对称加密算法（如 AES），但缺乏动态密钥管理机制，容易受到重放攻击的威胁。传统的 MD5 加密不足以达到足够高的安全性。因此这部分可以尝试使用基于 MD5 和 SHA-256 的多重哈希方法，用于提升数据完整性验证的安全性，也特别适用于在线考试这种多文件传输和存储的场景。^[4]

考中监控功能多以简单的屏幕截图或视频录制为主，实时性差，无法有效检测考生的违规操作。

目前国内主要采用的大多是牛客在线考试系统以及部分竞赛采用的 OMS 监考系统。这些线上考试平台，通常支持很多监考模式，例如最基本的强制全屏沉浸模式、进程检测监考以及双机位监考，同时题型支持也比较完备，从常见的填空选择到技术向的程序题目都支持。这些平台，对于早期常见的如网络拥塞、时间控制、身份验证等问题，都有相应的解决策略。^[5]

目前国内在其他领域采用 C/S 架构的方案有很多，例如有一款面向电力设备在线监测的应用程序，这使得管理和分析设备更为简单，一些铁路货运专线管理的一体化信息系统也采用了这种架构。^{[6][7]}

国外考试信息化起步较早，在一项研究中，就已经开发了一种使用电子标识符进行远程计算机用户认证的方法，基于微软标准实现的远程桌面协议 (RDP)，并通过虚拟通道保护通信信道的安全性，^[8]同时，也提出了利用机器学习从 Windows RDP 事件日志中检测横向移动攻击的方法，并展示了所提模型在分类 RDP 会话方面的优越性能。^[9]在被广泛采用的在线考试平台中，以 ProctorU 和 Examity 为代表的在线考试平台在安全性和实时性方面具有领先优势：

ProctorU：通过实时屏幕共享和 AI 驱动的行为分析技术，可实时监控考生的屏幕操作和周围环境，及时检测作弊行为。

Examity：采用多层次身份验证（如生物识别和行为模式分析）和加密数据传输技术，确保考生信息的安全性。

目前，对于保证线上考试的公平，通常有且不限于以下几种解决方案：

1) 多层身份验证

作为最基础的防作弊措施，多层身份验证能一定程度上确保参与考试的是本人，例如，部分学校通常提供账号密码登录系统，并安排人工核验认证比对等。有时，在考试进行中系统还会随机抽检进行人脸识别，防止中途更换考生。

2) 题目与选项随机化

上机考试时，在线考试平台会随机从题库中随机抽取题目，为每位考生生成个性化试卷，每位考生所看到的题目顺序与选项顺序均为随机的，一般用于降低合谋作弊的可能性。

3) 远程多机位监考

这是目前高规格考试频繁使用的措施，通常包含三路音视频监控，即 PC 录屏、电脑摄像头以及手机副摄像头，实现 360 度全方位监考。监考人员可以通过实时音视频监控全面观察考生的行为，包括面部表情、手部动作及周围环境，从而有效预防和检测作弊行为。

4) 算法自动检测

利用算法和人工智能技术分析学生的答题行为，及时发现可疑的作弊行为。例如，监考工具可以检测学生是否打开了非考试相关的网页或应用程序，同时记录如多个人脸、无人脸、切屏等异常情况。一旦发现作弊行为，系统会自动记录并提醒监考人员进一步调查。

此外，还不乏有诸如结合 RSA 加密与压缩隐写技术实现的一种高效的数据隐藏方案，通过构建一个预测 SSH 流量行为的模型，实现了对 SSH 通信模式的有效识别^[10]以及通过引入包间到达时间特征及生成对抗网络（WGAN-GP）算法来减少误报率等研究，共同促进提升上机考试的安全性。^{[11][12]}

接下来主要阐述的是基于 C/S 架构的技术实践案例。

1.2.2 基于 C/S 架构与对称加密的图书馆管理系统

目前，图书馆中对于一些图书下架的操作，依赖于上级教育部门或者是国家机密文件下发，传统的基于高校组织人员人工核查速度慢，请供应商协助可以提高效率，但是可能存在泄密的风险。^{[13][14][15]}

这种图书馆管理系统采用了基于 C/S 的架构，实现了对于部分不良图书的自动核查以及下架功能，同时，对于图书馆内部文件实现了动态加密，保密性强。

对于内部的机密文件，采用了对称加密技术，提高了数据的安全性。^[16]

这种设计思路下，对于每个高校的图书馆，都视作为一个客户端，图书馆中每本书的信息经由管理员录入软件，此时软件本地核查服务端下发的不良图书名单，自动筛选需要下架的图书，生成一份下架名单。

服务端则由各自图书馆的上级部门持有，服务端需要维护图书馆馆藏数据，负责各类图书的定性工作。

这种技术在成都师范学院图书馆有所应用，支持筛选有害图书，并建立了基于对称加密技术的 Oracle 服务端，实际落地了 C/S 系统架构的应用。

1.2.3 基于 C/S 模式的远程网络视频监控系统

传统的数字化模式的视频监控系统在安全性要求较高的场景略显乏力。^[17] 这种基于 C/S 模式的网络监控系统会将远程采集的数据，通过本地设备的加密运算，再使用特殊的传输通道，将加密后的数据传输到接收端，解密后再将数据传输到显示终端上。

在传输过程中，传统的视频编码不能很好的将数据压缩至合适的码率，导致在终端处显示的画面十分模糊，这里采用了基于 H.264 的编码技术，使得图像质量以及压缩率有了更好地表现。

H.264 是 MPEG-4 标准所定义的最新格式，是 MPEG-4 标准的第十部分，有时也称为 AVC，^[18] 它的编解码流程主要包含有变换（Transform）和反变换、量化（Quantization）和反量化、环路滤波（Loop Filter）、熵编码（Entropy Coding）。^[19]

此外，该编码技术还采用了许多新技术，如帧内预测^[20]，帧间预测^[21]，多参考帧运动估计^[22]，1/4 像素和 1/8 像素精度预测^[23]，4×4 整型变换等技术^[24]，使得编码效率加快，在保证图像质量的情况下增加了视频压缩码率。

1.3 论文组织结构

全文内容共六章，具体内容组织如下：

第一章为引言。

第二章为关键技术概述。

第三章为设计需求分析，得出本设计的方案。

第四章为系统设计与实现，介绍关键功能的实现方案。

第五章为系统测试，主要展示系统的兼容性。

第六章总结与展望，总结了本设计的具体工作，展望了后续可以精进的细节。

2 关键技术概述

2.1 C/S 软件体系

C/S（客户端/服务器）软件体系是一种经典的网络架构模式，它将软件分为客户端和服务端两部分，通过明确的职责分离实现系统功能。客户端主要负责用户界面呈现、交互逻辑处理和部分数据验证，而服务端则集中处理核心业务逻辑、数据存储管理和安全控制。这种架构模式自 20 世纪 80 年代末期开始流行，经过多年发展已成为企业级应用的主流架构之一。

在技术实现层面，C/S 架构通常采用 Socket 通信或 RPC（远程过程调用）等方式实现客户端与服务端之间的数据交换，并通过特定的应用层协议规范数据格式。这种架构因其高效的数据传输机制和强大的交互能力，在许多领域得到了广泛应用，如企业资源规划（ERP）系统、客户关系管理（CRM）系统、在线游戏、数据库管理工具等。随着技术的发展，C/S 架构也在不断地与云计算、大数据、微服务等新技术融合，以适应新的应用场景和需求。

这种类型的软件通常具备以下特性：

- **交互性强：**C/S 架构允许客户端与服务端进行实时的数据交互，支持复杂的用户界面和丰富的交互模式，提高了系统的响应速度和用户体验。客户端可以在本地进行部分数据处理，减轻服务器负担。
- **数据安全性高：**核心数据和业务逻辑集中存储在服务端，便于统一管理和保护，可以实现更严格的访问控制和权限管理，降低数据泄露风险。服务器可以实施更复杂的安全策略，如数据加密、访问审计等。
- **良好的扩展性：**客户端和服务端可以独立部署和升级，服务器可以通过集群方式横向扩展以支持更多用户，具有良好的可伸缩性。系统可以根据业务需求灵活调整资源分配。
- **网络资源利用率高：**相比于 B/S 架构，C/S 架构可以更有效地利用网络带宽，通过优化的数据传输协议和本地缓存机制，减少不必要的网络通信，适合处理大量数据的应用场景。
- **安装和维护成本高：**需要在用户的电脑上安装专用的客户端程序，增加了部署和升级的复杂性。当系统需要更新时，可能需要同时更新所有客户端，管理难

度较大。

- **跨平台兼容性挑战：**不同操作系统平台的客户端需要分别开发和维护，增加了开发成本和技术复杂度，不利于跨平台使用。这也是近年来许多应用转向 B/S 架构的主要原因之一。

随着技术的演进，传统 C/S 架构也在不断发展，出现了富客户端（Rich Client）、胖客户端（Fat Client）和瘦客户端（Thin Client）等变体形式，以及混合型架构如三层架构（Three-tier Architecture）和分布式架构（Distributed Architecture）等，进一步提升了系统的灵活性和适应性。在当代软件开发中，C/S 架构虽面临 Web 应用的挑战，但在特定领域如高性能计算、复杂业务处理、实时交互等场景中仍具有不可替代的优势。

2.2 Electron 技术简介

Electron 是由 GitHub 开发的一个开源框架，允许开发者使用 Web 技术（如 JavaScript、HTML 和 CSS）来构建跨平台的桌面应用程序。

其核心思想是将 Chromium 和 Node.js 集成在一起，使得开发者能够在桌面端实现与 Web 一致的界面和交互体验。这样的架构使得 Electron 应用可以在 Windows、macOS 和 Linux 等多个平台上运行，具有很高的跨平台兼容性，同时也具备强大的本地功能和性能，不仅弥补了 Web 应用在桌面端的不足，也使得客户端的跨平台开发变得更加容易。

Electron 的主要优势在于：

- **跨平台兼容性强：**同一套代码可以在 Windows、macOS 和 Linux 等主流操作系统上运行，极大地降低了开发和维护成本。
- **开发效率高：**开发者可以利用现有的前端技术栈和丰富的生态资源，快速搭建桌面应用。
- **集成 Node.js 能力：**Electron 应用不仅可以访问本地文件系统，还能调用各种 Node.js 模块，实现更强大的本地功能。
- **自动更新与打包：**Electron 提供了自动更新、应用打包和分发等完整的解决方案，便于产品的持续迭代和用户体验提升。

在本系统中，Electron 作为客户端的核心开发框架，承担了考试监控工具的

界面渲染、数据采集、加密存储等关键任务。通过 Electron，可以实现对本地屏幕、摄像头等硬件资源的访问，并结合前端技术实现友好的用户交互界面。此外，Electron 的安全机制和灵活的插件体系，也为系统的扩展性和安全性提供了有力保障。

2.3 Node-Media-Server 流媒体服务器

NodeMediaServer 是基于 Node.js 实现的轻量级流媒体服务器，它支持 RTMP、HTTP-FLV、WebRTC 等主流流媒体协议。它采用模块化设计，具有高度的可扩展性和定制化能力，是当前开源流媒体服务器中性能表现较为出色的解决方案之一。在本系统中，它作为客户端的本地流媒体服务器，负责接收和处理来自 FFmpeg 的视频流数据，并提供稳定的流媒体分发服务。它采用模块化设计，具有高度的可扩展性和定制化能力，是当前开源流媒体服务器中性能表现较为出色的解决方案之一。在本系统中，它作为客户端的本地流媒体服务器，负责接收和处理来自 FFmpeg 的视频流数据，并提供稳定的流媒体分发服务。

其主要特点包括：

- **低延迟传输：**通过 RTMP 协议实现毫秒级的视频流传输，保证监考画面的实时性。在实际测试中，端到端延迟可控制在 200-500ms 范围内，满足实时监控的严格要求。同时，支持 HTTP-FLV 协议，可在网络条件不佳时提供更稳定的传输能力。
- **跨平台支持：**基于 Node.js 开发，可在 Windows、Linux、macOS 等多个平台上运行，与 Electron 应用完美配合。这种跨平台特性使得监考系统可以在不同操作系统环境下保持一致的性能表现，大大降低了部署和维护成本。
- **资源占用低：**采用事件驱动的非阻塞 I/O 架构，即使在处理多路视频流时也能保持较低的系统资源占用。在实际应用中，单核处理器即可支持数十路 720p 视频流的并发处理，非常适合在考生端本地部署使用，不会明显影响考试过程中的系统性能。
- **配置灵活：**支持自定义端口、转码参数、认证机制等配置，方便根据实际需求进行调整。在监考系统中，可以通过配置文件动态调整视频质量和带宽占用，适应不同网络环境的需求。
- **安全机制完善：**提供基于 Token 的访问控制和 HTTPS 加密传输支持，确保视频流数据的安全性。在监考场景中，可以为每个考生生成唯一的访问令牌，防

止未授权访问和数据泄露。

- **集群扩展能力：**支持通过 Redis 实现多节点集群部署，可以根据监考规模动态扩展系统容量。这一特性为大规模考试提供了技术保障，理论上可支持数千考生同时在线监控。

在监考系统的实际应用中，Node-Media-Server 作为本地流媒体服务的核心组件，不仅负责接收来自 FFmpeg 的屏幕和摄像头视频流，还需要对视频流进行必要的处理（如添加水印、调整码率等）后再分发给监考服务器。其高效的流处理能力和稳定的性能表现，为实时监考提供了可靠的技术支持。

2.4 FFmpeg 音视频处理工具

FFmpeg 是一个功能强大的开源音视频处理工具集，由 FFmpeg 项目组于 2000 年开始开发，目前已成为业界最为广泛使用的多媒体处理框架。它提供了完整的音视频编解码、转码、混流、滤镜和推流等功能，支持几乎所有常见的音视频格式。在本系统中，FFmpeg 主要用于屏幕录制和摄像头画面的采集与推流，是实现实时监考功能的核心组件之一。

从技术架构上看，FFmpeg 主要由以下几个核心组件构成：

- **libavcodec：**提供了各种音视频编解码器的实现，支持 H.264、H.265、VP9 等主流视频编码格式。
- **libavformat：**负责音视频封装格式的处理，如 MP4、FLV、TS 等容器格式的解析与生成。
- **libavfilter：**提供丰富的音视频处理滤镜，可实现画面叠加、裁剪、缩放等复杂效果。
- **libavdevice：**用于访问各种输入输出设备，如摄像头、麦克风、屏幕等。
- **libswscale：**提供高效的图像缩放和色彩空间转换功能。

在监考系统中，FFmpeg 的主要应用特性包括：

- **多源采集与合成：**支持同时采集屏幕画面和摄像头视频，并通过高级滤镜链（filtergraph）进行画中画合成处理。在监考场景中，可以将摄像头画面作为小窗口叠加在屏幕画面上，实现对考生行为和屏幕内容的同步监控。具体实现

上，可以使用 FFmpeg 的 overlay 滤镜完成画面合成，示例命令如下：

```
ffmpeg -f gdigrab -framerate 15 -i desktop \
    -f dshow -i video="Webcam" \
    -filter_complex "[1:v]scale=320:-1[cam]; \
        [0:v][cam]overlay=main_w-320:0" \
    -c:v libx264 -preset ultrafast \
    -tune zerolatency \
    -f flv rtmp://localhost/live/stream
```

- **实时编码与优化：**采用 H.264 编码，并针对实时性场景进行特殊优化。在监考系统中，视频质量和传输延迟需要进行平衡，通常采用以下参数设置：
 - preset=ultrafast: 最快的编码速度，降低编码延迟
 - tune=zerolatency: 优化低延迟传输场景
 - profile=baseline: 提高兼容性，降低解码复杂度
 - keyint=30: 每秒 1 个关键帧，便于快速恢复和随机访问
 - crf=23: 在文件大小和质量之间取得平衡的恒定质量因子
- **多协议推流能力：**支持 RTMP、RTSP、HLS、SRT 等多种流媒体协议，可根据网络环境和应用需求灵活选择。在本系统中，主要采用 RTMP 协议将处理后的视频流推送至本地 Node-Media-Server，再由服务器进行分发。RTMP 协议具有低延迟、实时性好的特点，非常适合监考场景的实时视频传输需求。
- **硬件加速：**支持 NVIDIA NVENC、Intel QuickSync、AMD AMF 等硬件编码技术，大幅降低 CPU 占用率。在实际应用中，通过启用硬件加速，可以将视频编码的 CPU 占用从 30-40% 降低到 5-10%，有效减少对考生电脑性能的影响。系统会根据考生电脑的硬件配置自动选择最适合的加速方案：

```
# NVIDIA GPU 加速示例
ffmpeg ... -c:v h264_nvenc -preset p1 -tune ll ...

# Intel GPU 加速示例
ffmpeg ... -c:v h264_qsv -preset veryfast ...
```

- **网络自适应：**通过动态调整码率和分辨率，适应不同的网络环境。在带宽受限

的情况下，系统可以自动降低视频质量，确保监考画面的连续性和实时性。这一特性对于保障各种网络环境下的监考效果至关重要。

- **错误恢复机制：**内置了完善的错误检测和恢复机制，在网络波动或设备异常时能够快速恢复推流。系统会定期检测推流状态，在发现异常时自动重启 FFmpeg 进程，最大限度地减少监控中断时间。

在监考系统的实际部署中，FFmpeg 作为独立进程运行，由 Electron 应用通过进程间通信进行控制和管理。这种架构设计既保证了视频处理的高效性，又避免了因视频处理导致的主应用卡顿，为监考系统提供了稳定可靠的音视频处理基础。

2.5 WebSocket 实时通信技术

WebSocket 是一种在单个 TCP 连接上进行全双工通信的协议，由 IETF 在 RFC 6455 中标准化，设计用于解决传统 HTTP 请求-响应模式在实时通信场景中的局限性。WebSocket 协议在 2011 年成为标准，目前已被广泛应用于即时通讯、在线游戏、金融交易等需要低延迟双向通信的场景。在本监考系统中，WebSocket 作为核心通信技术，用于实现客户端与服务器之间的实时数据交换，确保监考过程的实时性和可靠性。

2.5.1 WebSocket 协议原理

WebSocket 协议建立在 HTTP 协议之上，通过一次 HTTP 握手来完成连接的建立，之后便转换为基于帧的轻量级协议进行通信。其工作流程主要包括以下几个步骤：

1. **连接建立：**客户端通过 HTTP 发送特殊的升级请求（Upgrade: websocket），服务器接受后返回状态码 101，表示协议切换成功。
2. **数据传输：**连接建立后，双方可以通过该连接发送文本或二进制数据帧，无需额外的连接建立过程。
3. **心跳机制：**WebSocket 协议内置了 Ping/Pong 帧用于连接保活，确保连接的稳定性。
4. **连接关闭：**任何一方都可以发送关闭帧来终止连接，包含关闭状态码和原因。

在数据格式上，WebSocket 支持文本和二进制两种基本数据类型，并可通过

应用层协议（如 JSON、Protocol Buffers 等）实现更复杂的数据结构传输。相比于 HTTP 请求，WebSocket 帧头部更加精简，通常只有 2-14 字节，大大减少了通信开销。

2.5.2 技术优势与特点

在实时监控系统中，WebSocket 技术具有以下关键优势：

- **持久连接与低延迟：**建立连接后可持续通信，避免频繁的连接建立和断开，消息传递延迟通常在 50ms 以内，远低于传统 HTTP 轮询的 300-500ms。这一特性确保了监考指令和状态更新能够及时传达，对于检测和响应考试异常行为至关重要。
- **双向实时通信：**服务器可以主动向客户端推送消息，无需客户端轮询请求。在监考场景中，这使得监考人员能够实时下发指令（如要求考生调整摄像头、停止可疑行为等），同时考生端也能立即报告异常状况（如网络波动、设备故障等）。
- **资源占用低：**相比 HTTP 轮询，WebSocket 大幅减少了网络带宽和服务器资源的占用。实际测试表明，在 1000 名考生同时在线的场景下，WebSocket 方案的服务器 CPU 占用率比 HTTP 轮询低约 60%，网络流量减少约 70%。这一优势使得系统能够支持更大规模的考试监控，同时降低了基础设施成本。
- **跨平台兼容性：**所有现代浏览器和主流开发框架（包括 Electron）均原生支持 WebSocket 协议，确保了系统在不同平台和环境下的一致性表现。在本系统中，服务端使用 Node.js 的 ws 库实现 WebSocket 服务器，客户端则利用 Electron 内置的 WebSocket API 进行通信，实现了无缝集成。
- **安全性保障：**WebSocket 支持 WSS（WebSocket Secure）协议，基于 TLS/SSL 加密通信内容，防止数据被窃听或篡改。在监考系统中，所有 WebSocket 连接均采用 WSS 协议，并结合 JWT（JSON Web Token）实现身份验证和会话管理，确保通信安全。
- **可扩展性：**WebSocket 协议支持子协议扩展，可以根据应用需求定制通信规则。在监考系统中，我们定义了专用的消息格式和事件类型，用于处理不同类型的监考数据和指令，提高了系统的灵活性和可维护性。

2.5.3 在监考系统中的应用

在本监考系统的实际应用中，WebSocket 技术主要用于以下几个关键场景：

- **考试状态同步：**实时传输考试进度、剩余时间等状态信息，确保考生和监考人员对考试状态有一致的认知。系统每 5 秒自动同步一次考试状态，同时在关键节点（如开始考试、结束提醒等）即时推送通知。
- **实时指令下发：**监考人员可以通过 WebSocket 通道向特定考生或全体考生发送指令，如锁定屏幕、禁用特定功能、要求调整摄像头等。这些指令通过优先级队列管理，确保关键指令能够优先处理。
- **异常事件报告：**当检测到可疑行为（如切换应用程序、插入外部设备、摄像头遮挡等）时，客户端会立即通过 WebSocket 向服务器报告，并附带相关证据（如屏幕截图、日志记录等）。系统采用了基于令牌桶的流量控制算法，防止异常事件报告过于频繁导致服务器过载。
- **流媒体控制：**通过 WebSocket 通道控制视频流的启动、停止和参数调整，实现对监控画面的动态管理。例如，在网络条件变差时，可以通过 WebSocket 指令降低视频码率和分辨率，确保监控的连续性。
- **系统健康监测：**客户端和服务器之间通过 WebSocket 定期交换心跳消息，监测连接状态和系统健康度。如果检测到连接异常，系统会自动尝试重连，并记录断连时间和原因，为后续分析提供依据。

在现代的处理器上，通过 WebSocket 通信是非常高效的。在这种设计思路下，客户端可以近乎实时地调整推流的码率与清晰度，同时支持高效的监考指令传达与响应。此外，这种 WebSocket 设计有着良好的扩展性和灵活性，能够满足不同场景下的需求。

2.6 本章小结

本章节主要介绍了开发本系统时会涉及到的相关技术。

3 需求分析

在开始开发本系统之前，需要明确有哪些涉及到人员方面的需求，最先要明确的是考生的需求，而在考试之外，会涉及到考前配置软件的需求。

上述是涉及到人员方面的需求，接下来是涉及到技术方面的需求。

首先，本系统主要的功能是监考，因此，有屏幕捕获与推流的需求，其次，对考试机的系统监控与考中指令的下发也是一个需求，最后，也就是系统的安全措施相关需求。

而本系统的服务端部分，则含有对用户的认证与授权的需求，直播流管理的需求和考中指令的下发与处理的需求。

3.1 涉考人员相关需求分析

首先需要明确考生实际的会接触到的场景和其需求，考虑到参考人员一般只是前来参加考试，而不涉及其他部分，在客户端方面的设计就应该简单易上手，尽量避免复杂的交互操作。因此，在正常流程下，考生只涉及到登录系统这一个流程。

考虑到系统的泛用性，通信服务器参数应该是可变的，所以服务器通信地址、访问密码等等都应设计成可调参数，因此，在客户端方面，需要提供一个服务器配置界面，方便志愿者进行考前配置。

3.2 功能需求分析

考前服务器参数配置以及考生登录需要提供 GUI 界面，方便用户交互，在完成这两个流程之后，本系统就应最小化到系统托盘后台运行，不影响考生正常考试。

当本系统进入到后台，也就是考生正常登录之后，本系统就应开始自动建立一个本地直播服务器，并实时捕获考试机屏幕上的所有内容以及摄像头的内容（如果有），同时维护一个 WebSocket 链接，方便监考指令的下发，此外，考试机的流地址也会提供给服务器。

考虑到远程监考需要一些诸如：MAC 地址、IP 地址、CPU 使用率等的系统信息来辅助监考员判断考试机状态，所以，本系统还需要一个默认的系统信息采集服务，并定期发送系统信息到服务器，同时，这部分也可以根据服务器的需求自由调整来采集额外的信息。

在安全性相关的需求方面，本系统应不允许考生随意关闭或终止程序，关闭操作需要输入特定密码（由服务端验证）才能执行，此外，系统还需要防止多实例运行，确保监控的唯一性和持续性。

3.3 服务端需求分析

服务端作为整个监控系统的管理中心，负责汇聚和处理来自各客户端的数据流，并提供统一的监控界面。其主要功能需求包括：

1. 客户端连接管理

服务端需维护与各客户端的连接状态，并对客户端的接入请求进行身份验证和授权管理。系统应能实时显示所有在线客户端的基本信息，便于监考人员掌握考场整体状况。

2. 视频流接收与分发

服务端需接收并汇总来自客户端的流地址，提供选择界面让监考人员选择查看特定考生的监控画面。同时，支持将选中的视频流转发至第三方直播工具（如 OBS），便于监考团队协作监控。

3. 消息通知管理

服务端应提供向单个或全体考生发送通知的功能，支持设置消息优先级和显示方式。同时，记录所有通知的发送历史，便于后续审查和分析。

4 系统设计与实现

本系统采用 C/S 架构设计，分为客户端和服务端两个主要部分。客户端基于 Electron 框架开发，利用 Node.js 的底层能力和 Chromium 的界面渲染能力，负责考试机的屏幕捕获、视频推流和系统监控；服务端基于 Golang 开发，采用 Gin 框架实现 Web 服务，负责客户端管理、视频流分发和监考指令下发。

4.1 客户端设计与实现

客户端采用模块化设计，基于 Electron 构建，整合了 Node.js 的后端能力与 Chromium 的前端界面展示能力。其核心逻辑主要在 Electron 的主进程中实现，负责应用的生命周期管理、系统交互以及与渲染进程的通信。客户端主要包含以下几个核心模块：

4.1.1 主进程与应用管理模块

此模块是客户端应用的入口点和核心控制器，基于 Electron 的 `app` 和 `BrowserWindow` 模块构建。

应用生命周期管理：通过监听 `app` 对象的生命周期事件来管理应用状态。例如，在 `app.on('ready', callback)` 事件触发后，完成应用的初始化，包括创建主窗口 (`BrowserWindow`)、加载渲染进程的 HTML 文件以及创建系统托盘图标。通过 `app.requestSingleInstanceLock()` 方法确保系统只运行一个客户端实例，防止多开影响监控。为确保应用在所有窗口关闭后仍能在后台运行（特别是进行推流和监控），此模块还监听了 `app.on('window-all-closed', callback)` 事件，也就是退出软件的事件，它阻止了默认的退出行为，除非用户通过托盘菜单输入退出密码显式退出。

本系统客户端生命周期如图 4.1 所示。

窗口与交互管理：创建的主窗口承载了用户交互界面（如登录、状态显示）。为了防止考生意外或故意关闭监控，主窗口的关闭行为被拦截：监听窗口的 `close` 事件，并通过 `event.preventDefault()` 阻止窗口关闭，转而调用 `mainWindow.hide()` 将窗口隐藏，同时依赖系统托盘 (`Tray`) 进行后续交互。系统托盘图标使用 `Tray` 模块创建，并关联了一个上下文菜单 (`Menu.buildFromTemplate`)，提供“显示窗口”和需要密码验证的“退出程序”选项。

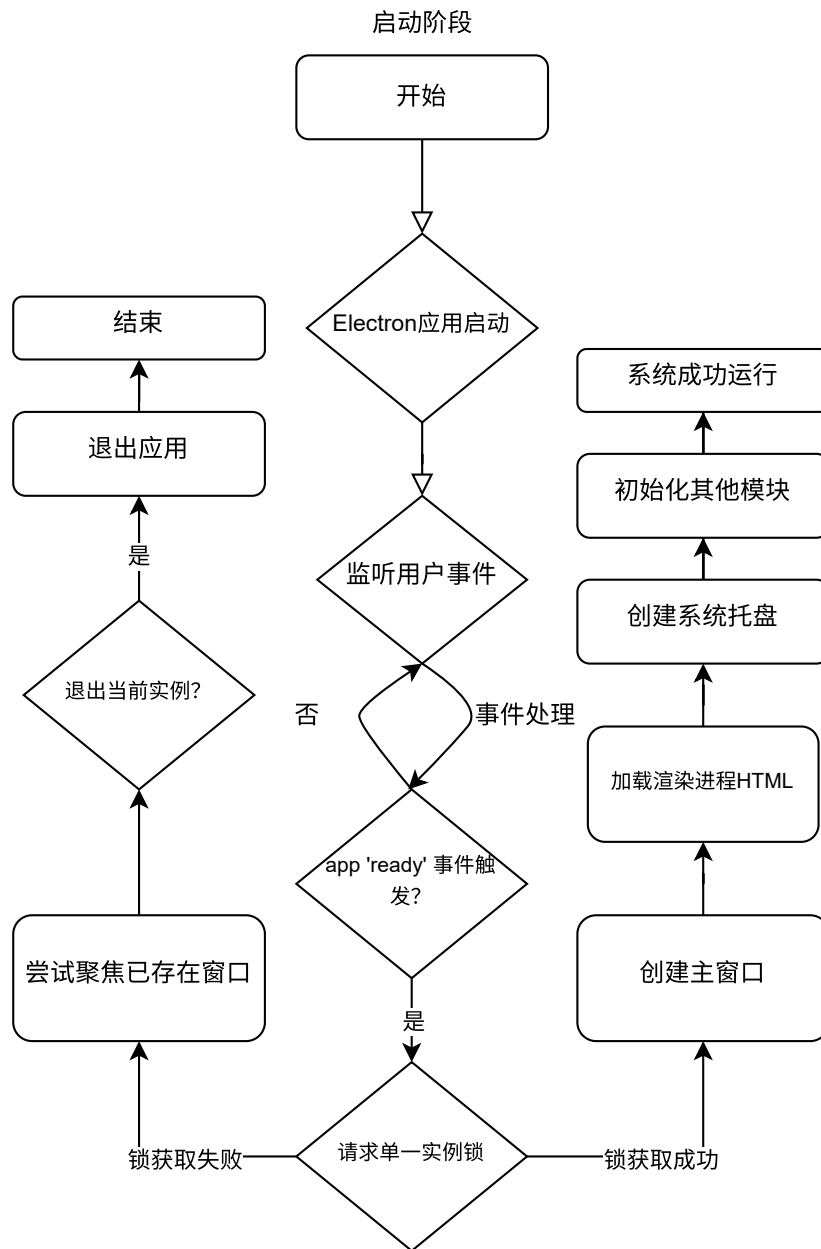


图 4.1 客户端应用生命周期管理流程图示例

进程间通信: 主进程与渲染进程（负责 UI 展示）之间通过 Electron 的进程间通信机制 (ipcMain 和 ipcRenderer) 进行交互，这样可以缓解因事件阻塞时导致的软件“卡死”现象。例如，渲染进程通过 `ipcRenderer.send(channel, ...args)` 发送登录信息或配置信息，主进程通过 `ipcMain.handle(channel, listener)` 或 `ipcMain.on(channel, listener)` 接收并处理这些请求，如验证配置、启动服务等。反之，主进程也可以通

过 `mainWindow.webContents.send(channel, ...args)` 向渲染进程发送状态更新或服务器指令。

4.1.2 媒体服务模块

该模块是实现考试监控的核心，负责屏幕和摄像头画面的采集、编码与推流。

视频采集与处理：本系统通过调用跨平台的 FFmpeg 技术，并结合 Node.js 来创建一个独立的 FFmpeg 子进程来执行采集和推流任务，避免了复杂的 C++ 绑定和潜在的性能瓶颈。此外，在不同的操作系统平台上，动态构建 FFmpeg 启动命令，来适配多平台的不同使用方式。

编码与推流：在调用 FFmpeg 时，本系统会尝试采用硬件编码来降低延迟和系统占用，生成的本地流指向 Node 的媒体服务器。推流使用的 `streamKey` 则是与考生的唯一标识关联。客户端代码负责管理 FFmpeg 子进程的生命周期，监听其各类事件。通过解析 FFmpeg 输出可以获取推流状态和错误信息，并在进程意外退出时尝试自动重启 FFmpeg 进程以保证监控的连续性。

本地流媒体服务：客户端依赖于一个在本地运行的 Node-Media-Server (NMS) 实例。FFmpeg 将处理后的视频流推送到本地 NMS。NMS 负责接收这个本地 RTMP 流，在本系统中提供其他可用的如 HTTP-FLV 或其他协议的转播地址。这样做的好处是将流处理的复杂性与客户端核心逻辑解耦，并且 NMS 相对稳定。客户端随后会将这个本地流的播放地址（如：`http://127.0.0.1:8000/live/streamKey.flv`）报告给服务端。

4.1.3 API 服务与通信模块

此模块负责客户端与考试服务器之间的通信请求，其中，对于实时性要求较高的监考指令下发与考试机系统信息部分，采用了 WebSocket 协议。

WebSocket 连接管理：使用 `ws` 库创建 WebSocket 客户端实例。根据配置管理模块提供的服务器地址 (`ws://server-address:port` 或 `wss://` for secure connection) 发起连接。通过监听 `open`, `message`, `error`, `close` 等事件来管理连接状态。`open` 事件表示连接成功建立，此时客户端会发送身份验证信息（如携带 `token` 或考试机编号）。

消息处理与交互：`message` 事件的回调函数负责接收来自服务器的消息。这

部分的消息采用 JSON 格式，客户端收到后使用 `JSON.parse()` 解析。根据消息中的 `type` 字段或类似标识，执行不同操作，例如：处理服务器下发的监考指令（如 `lockScreen`, `showWarning`, `requestSystemInfo`）、响应心跳包、更新本地状态等。客户端也主动通过 `WebSocket` 发送消息，例如：定期发送心跳包、上报系统基本信息、报告本地推流地址、响应服务器指令的结果等。所有发送的消息同样被格式化为 JSON 字符串 (`JSON.stringify()`)。

系统信息采集与上报：利用系统库定期采集考试机的关键运行状态。通过调用该库提供的异步函数，如 `si.cpu()`, `si.mem()`, `si.networkStats()`, `si.processes()` 等，获取 CPU 使用率、内存占用、网络流量、活动进程列表等信息。采集任务通过 `setInterval` 定时触发（例如每 30 秒一次），并将获取到的信息通过 `WebSocket` 发送给服务器，供监考人员参考或用于异常行为检测。

断线重连机制：在 `WebSocket` 的 `close` 或 `error` 事件触发时，表明连接中断。客户端实现了自动重连逻辑：在本系统中，使用了 `setTimeout` 设置一个延迟，然后重新尝试建立 `WebSocket` 连接。这种机制确保了在网络暂时不稳定的情况下，客户端能够自动恢复与考试服务器的通信。

4.1.4 配置管理模块

该模块负责存储和管理客户端运行所需的配置信息，如服务器地址、认证凭证、考试机标识等。

配置存储：使用 `electron-store` 库进行本地配置的持久化存储。该库提供了一个简单的键值对 API (`store.set(key, value)`, `store.get(key)`)，并将数据存储为用户操作系统的标准应用配置目录下。

配置加密：为了增强安全性，存储在本地的敏感配置信息（特别是服务器访问凭证）进行了加密处理。本系统采用 Node.js 内置的 `crypto` 模块实现 AES-256-CBC 对称加密。在存储配置前，使用 `crypto.createCipheriv(algorithm, key, iv)` 创建加密器对敏感值进行加密；在读取配置后，使用 `crypto.createDecipheriv(algorithm, key, iv)` 进行解密。

配置加载与验证：应用启动时，首先会尝试从 `electron-store` 加载配置。如果配置不存在或不完整，将会引导用户进入配置界面（通过渲染进程实现）进行输

入。加载配置后，会对关键信息（如服务器地址格式）进行基本验证。配置信息（特别是认证凭据）在建立 WebSocket 连接时发送给服务器进行最终验证。

4.2 服务端设计与实现

服务端作为整个监控系统的中心枢纽，负责管理所有接入的客户端、处理实时数据流、响应监考指令并提供管理界面。考虑到系统对实时性、稳定性和后续扩展性的要求，服务端采用 Go 语言进行开发，并选用 Gin Web 框架构建基础服务。整体架构倾向于模块化设计，便于功能迭代与维护。

服务端的核心功能围绕以下几个模块展开：

1. Web 服务模块 (基于 Gin)

此模块是服务端与外部交互的主要入口，负责处理 HTTP 请求。它不仅为监考人员提供管理界面所需的静态资源（HTML, CSS, JavaScript），还提供了一系列 RESTful API 接口，用于客户端认证、信息查询、状态管理等操作。Gin 框架以其高性能和简洁的 API 设计，能够有效处理高并发请求，并提供了中间件支持，便于实现日志记录、跨域处理 (CORS)、身份验证等通用功能。关键 API 接口功能示例如下表 4.1 所示。

表 4.1 服务端核心 API 接口示例

路径	方法	主要功能
/api/auth/login	POST	客户端或管理员登录认证
/api/clients	GET	获取在线客户端列表及状态
/api/clients/{id}/stream	GET	获取指定客户端的流媒体播放地址
/api/clients/{id}/info	GET	查询指定客户端的详细系统信息
/api/command	POST	向指定或全部客户端下发指令
/api/config	GET/POST	管理系统全局配置

2. WebSocket 服务模块

为满足监考过程中的实时通信需求（如指令下发、状态上报、心跳维持），服务端实现了 WebSocket 服务。此模块负责管理所有客户端的 WebSocket 长连接，处理双向数据帧。主要功能包括：维护连接池，跟踪客户端在线状态；高效地向单个、部分或所有客户端广播或定向推送消息（如监考指令、通知）；处理客户端上报的实时系统信息和异常事件；通过心跳机制检测并清理无效或断

开的连接，保证连接的稳定性。

3. 客户端管理模块

此模块负责维护客户端的整个生命周期信息。它记录了每个已连接客户端的标识、状态（在线、离线、异常）、登录时间、IP 地址、上报的系统信息等。当客户端通过 WebSocket 连接或断开时，该模块会更新其状态。此外，它还可能包含一些基础的异常行为检测逻辑（如检测重复登录、长时间无心跳等），并将相关信息展示给监考人员或触发告警。

4. 流媒体管理模块

该模块与客户端的媒体服务模块对应，核心职责是管理所有客户端上报的流媒体地址。它接收客户端报告的本地流地址（如 HTTP-FLV 地址），并将其与客户端标识关联存储。监考人员通过 Web 界面请求查看某个考生的监控画面时，Web 服务模块会调用此模块获取对应的流地址，并返回给前端播放器。此模块还需监控流状态的有效性（例如，通过定期探测或接收客户端更新），并可能支持将流地址转发给第三方平台（如配置了推流地址）。

这些模块协同工作，共同构成了服务端的完整功能。通过合理的模块划分和技术选型，旨在构建一个稳定、高效且易于管理的远程监考系统后端。

4.3 线下考试无纸化密码分发

除了通用的功能需求外，针对特定的考试场景，认证和配置流程需要特别考虑。以常见的线下机房考试（如 C 语言期末上机考试、XCPC 程序设计竞赛）为例，如何在保障安全性的前提下，高效地完成大量考试机的登录认证是一个挑战。

市面上针对此类线下场景，存在多种密码或凭证分发方式：

- **纸质分发：**将包含登录账号密码的纸条提前放置在每个座位上，或在考试开始时统一分发。优点是简单直接，但缺点是准备工作繁琐，纸条易丢失或被他人窥视，安全性较低，且不够环保。
- **现场口令：**监考员在考试开始时通过口头或白板告知统一的登录密码。这种方式仅适用于密码一致且安全性要求不高的场景，且容易造成现场混乱和信息泄露。
- **预设账号：**管理员提前为每台机器配置好特定的考试账号和密码，甚至直接登录。这种方式对管理员工作量要求大，且如果密码简单或固定，存在安全隐患。

考虑到这些方案的优缺点以及线下考试通常会有志愿者或工作人员提前布置考场的实际情况，本系统在设计客户端时考虑了一种结合人工配置的优化流程：考前，负责配置考试机的志愿者或工作人员，在安装和设置客户端软件（如配置服务器地址）的同时，可以利用其权限获取或设置该场考试所需的登录凭证（例如，一个临时的、与机器或座位绑定的密码/Token）。志愿者可以直接使用该凭证完成客户端的登录认证过程。这样，当考生到达指定座位时，考试监控客户端已经处于登录并就绪的状态，考生只需关注考试本身即可，无需进行额外的登录操作。

该方案的优势在于：

- **简化考生操作：**考生无需记忆和输入密码，降低了考前准备的复杂度，减少了因密码输入错误导致的问题。
- **提高安全性：**相较于直接将密码暴露给考生，由受信任的志愿者操作，可以在一定程度上减少密码在开考前泄露的风险。
- **利用现有流程：**将认证步骤融入到志愿者本就需要进行的考前机器配置流程中，没有增加额外的角色或复杂的系统。

然而，该方案也存在局限性：它强依赖于线下有足够的人力（志愿者或工作人员）来完成配置和预登录；对于完全远程的线上考试场景，这种模式显然不适用。

因此，对于线上远程考试，仍需考虑其他认证方式，例如：通过安全的邮件系统向考生预发送一次性密码或登录链接；利用手机扫描二维码进行身份验证和登录；集成现有的单点登录（SSO）系统等。这些方案各有优劣，需要在安全性、易用性和实施成本之间进行权衡。

展望未来，结合生物识别技术（如人脸识别登录）或更安全的无密码认证方案（如 FIDO/WebAuthn），可能是进一步提升考试登录环节安全性和便捷性的发展方向。

5 系统测试

系统测试阶段旨在验证所开发的远程监考系统是否满足设计需求，具备预期的功能、性能、稳定性和安全性。考虑到项目的实际进度和资源，本阶段的测试工作主要集中在 Windows 操作系统环境，辅以部分功能的跨平台初步验证。测试方法结合了功能性测试、性能评估和基础安全检查。

5.1 测试环境配置

为模拟实际部署场景，测试在不同硬件配置的 Windows 10/11 系统上进行。本测试使用的主要硬件环境配置如表 5.1 所示。网络环境则模拟了常见的有线和无线连接，并在可能的情况下引入了延迟和丢包，以评估系统在不同网络条件下的表现。

表 5.1 主要测试环境配置

测试项	测试环境
操作系统	Windows 11 家庭版 24H2
处理器	AMD Ryzen 7 5800H
内存	16GB DDR4
显卡	NVIDIA GeForce RTX 3060 Laptop
网络环境	1000Mbps 有线/ 5Ghz Wi-Fi 无线
摄像头	720p 内置
并发客户端数	单一客户端

服务端的硬件配置如下：

表 5.2 服务端测试环境配置

测试项	测试环境
操作系统	macOS Sequoia 15.4
处理器	Apple M1 Pro
内存	16GB 统一内存
显卡	Apple M1 Pro
网络环境	300 Mbps Wi-Fi 无线
并发客户端数	模拟 300 路客户端

5.2 功能测试

功能测试旨在验证系统客户端和服务端的各个模块是否按照设计需求正常工作。测试覆盖了用户界面交互、后台服务运行、数据采集与传输、指令接收与执行等关键功能点。测试在 Windows 11 环境下进行。

客户端功能测试：客户端的功能测试主要关注其在考试机上的行为表现和数据采集传输的准确性。关键测试项包括：

- **应用生命周期管理：**测试了应用的启动（单一实例锁定）、窗口隐藏、系统托盘图标显示及菜单功能。验证了通过托盘菜单退出时是否正确触发密码验证流程。
- **配置管理：**测试了使用 electron-store 保存和读取配置信息的功能，并初步验证了配置文件的基础加密（混淆）是否生效，文件内容是否不易直接辨读。
- **媒体采集与推流：**验证了 FFmpeg 子进程是否能成功启动，并正确捕获屏幕和摄像头画面。检查了画面合成（画中画）效果和本地 Node-Media-Server (NMS) 是否能接收到 RTMP 推流。初步验证了在支持硬件加速的机器上，FFmpeg 参数是否正确包含了硬件加速选项。
- **本地流媒体服务：**测试了本地 NMS 是否能正常运行，并提供如 HTTP-FLV 格式的转播地址。
- **API 服务与通信：**验证了客户端能否成功建立与服务端的 WebSocket 连接，并进行身份认证。测试了客户端定期上报系统信息（CPU、内存、进程列表等，通过 systeminformation 库采集）是否正常，以及能否正确接收和响应服务器下发的指令（如模拟锁定屏幕、显示消息等）。验证了断线后自动重连机制是否有效。

服务端功能测试：服务端功能测试主要侧重于其核心业务逻辑和与多个客户端交互的能力。关键测试项包括：

- **客户端连接管理：**测试了服务端能否接收新的 WebSocket 连接，进行客户端身份验证（基于机器编号和密码/Token），并将在线客户端添加到管理列表。验证了客户端断开连接时，服务端能否及时检测并更新其在线状态。
- **Web 服务接口：**测试了主要的 RESTful API 接口，如获取客户端列表、查询特定客户端信息等，验证接口响应是否正确，返回数据是否符合预期。

- **消息处理与指令下发：**验证了服务端能否接收客户端上报的系统信息和流地址，并将其与对应的客户端关联。测试了通过管理界面（模拟或实际前端）向单个或全体客户端发送指令，验证指令能否被客户端正确接收和执行。
- **流媒体地址管理：**验证了服务端能否接收并存储客户端上报的本地流地址，并在请求时返回正确的地址。

功能测试结果汇总：

表 5.3 系统功能测试结果摘要

测试模块	主要测试功能	测试结果
客户端 - 应用管理	启动、单实例、窗口隐藏、托盘	通过
客户端 - 配置管理	配置存储、基础加密、加载	通过
客户端 - 媒体服务	FFmpeg 启动、采集、合成、推流至 NMS	通过
客户端 - 本地 NMS	NMS 运行、提供 HTTP-FLV 流	通过
客户端 - API 通信	WebSocket 连接、认证、信息上报、指令接收、重连	通过
服务端 - 连接管理	WebSocket 连接接收、认证、状态更新	通过
服务端 - Web 服务	RESTful API 调用 (如获取列表)	通过
服务端 - 消息/指令	接收客户端上报、指令下发	通过
服务端 - 流管理	接收/分发流地址	通过

5.3 性能测试

性能测试旨在评估系统在典型负载下的资源占用情况和响应能力，重点关注客户端对考试机性能的影响以及服务端的并发处理能力。

客户端性能评估：在 Windows 测试环境下，运行客户端进行屏幕和摄像头采集推流时，监测了其对考试机 CPU、内存和网络带宽的占用。测试表明，在开启硬件加速的情况下（如使用 NVIDIA 显卡），客户端的 CPU 占用率通常能维持在较低水平（约 0.9% - 3.2%），内存占用稳定在 115.7 MB 左右。上行网络带宽主要取决于视频编码参数和屏幕活动情况，通常在 30 Mbps 范围内。这些数据表明，客户端在多数现代考试机上运行时，对正常考试操作的影响较小。长时间运行测试（这里大约运行了 3.5 小时）未发现明显的性能衰退或内存泄漏。

服务端性能评估：服务端性能测试主要评估其能稳定支持的并发客户端连接数。通过模拟多个客户端连接并保持通信，观察服务端在不同连接数下的 CPU、

内存和网络负载。初步测试显示，在表 5.1 所示的服务端配置下，系统能够稳定处理约 300 路并发客户端连接，此时服务端各项资源占用处于合理范围，指令下发和信息上报的延迟可控。当连接数进一步增加时，系统性能可能开始出现瓶颈，具体取决于服务器硬件和网络条件。

5.4 基础安全检查

本阶段进行的基础安全检查主要关注系统实现的安全功能是否按设计工作，而非针对复杂的攻击手段进行渗透测试。

- **客户端防退出：**验证了禁用系统快捷键（如 Alt+F4）和拦截窗口关闭事件是否生效，以及通过托盘菜单退出时密码验证是否正确。
- **单实例运行：**验证了尝试启动客户端的第二个实例时，新实例是否能被正确阻止并退出。
- **配置信息保护：**初步检查了存储的配置文件内容是否经过基础加密，不易直接读取。
- **身份认证：**验证了客户端和管理员通过 WebSocket 或 API 连接时，使用错误的凭证是否会被服务器拒绝。

这些基础检查表明，系统在设计层面的安全措施已初步实现并生效。

5.5 测试结论与展望

基于上述测试，本远程监考系统在 Windows 11 环境下实现了设计中的核心功能，包括客户端的屏幕/摄像头采集推流、系统信息上报、指令接收，以及服务端的客户端管理、消息通信和流地址分发。系统在典型负载下资源占用合理，能满足中小型规模的在线考试监控需求。基础安全检查也验证了防止非法退出和单实例运行等关键安全特性。

由于时间限制，本阶段主要测试在 Windows 环境进行。未来的工作应包括在其他关键操作系统（如 Ubuntu Linux）下进行更全面的功能、性能和兼容性测试，确保系统在不同平台上都能稳定可靠运行。此外，进一步的安全性评估，包括针对性的渗透测试和代码审计，也是提升系统鲁棒性的重要方向。在性能方面，可以进行更精细的调优和压力测试，以探索系统在高并发场景下的极限承载能力。

6 总结与展望

本系统主要还是一个相对基础的监考与推流系统，还存在很多不足和改进的地方，例如在反作弊方面还没做到自动化判定与非法操作的阻止，对于潜在的人员作弊暂时没有很好的解决方法。尤其是本系统尚未经历实际情况下的高压使用，可能还存在许多潜在的问题。

目前具备了基本的功能，对监考系统的部署进行了大量的简化与自动化，方便了考时的实时监控与竞赛的直播功能。但是在防范人员顶替作弊方面仍然没有一个通用的高效的解决方案，所以在相关的反作弊方面仍然是一个值得继续探索的问题。

参考文献

- [1] AGGHEY A Z, MWINUKA L J, PANDHARE S M, et al. Detection of username enumeration attack on ssh protocol: Machine learning approach[J]. Symmetry, 2021, 13.
- [2] ZHANG C, QI D. How to parallelize rsync[J]. Journal of Physics: Conference Series, 2020, 1684.
- [3] FANG C, COTTRELL R L A, KISSEL E. When to use rsync[R]. Zettar Inc, Mountain View, CA (United States); SLAC National Accelerator Lab., Menlo Park, CA (United States); Lawrence Berkeley National Lab. (LBNL), Berkeley, CA (United States), 2021.
- [4] REDDY G P, NARAYANA A, KEERTHAN P K, et al. Multiple hashing using sha-256 and md5[C]//THAMPI S M, GELENBE E, ATIQUZZAMAN M, et al. Advances in Computing and Network Communications. Singapore: Springer Singapore, 2021.
- [5] 唐俊武, 南理勇, 左强. 在线考试系统开发中的几个问题及解决方法[J]. 计算机与数字工程, 2005(08): 144-147.
- [6] 邹宜金, 张美锋, 曹玲燕, 等. 基于 C/S 架构的电力设备数据库管理软件设计研究[J]. 电气技术与经济, 2023(04): 18-22.
- [7] 高厅权. 货运专线管控一体化系统的软件设计与实现[D]. 兰州交通大学, 2022.
- [8] KRAEV Y, FIRSOV G, KANDAKOV D. Authentication via rdp using electronic identifiers [C]//2021 IEEE Conference of Russian Young Researchers in Electrical and Electronic Engineering (ElConRus). 2021: 2361-2365.
- [9] BAI T, BIAN H, SALAHUDDIN M A, et al. Rdp-based lateral movement detection using machine learning[J]. Computer Communications, 2021, 165: 9-19.
- [10] LEE J, LEE H. Improving ssh detection model using ipa time and wgan-gp[J]. Computers and Security, 2022, 116.
- [11] WAHAB O F A, KHALAF A A M, HUSSEIN A I, et al. Hiding data using efficient combination of rsa cryptography, and compression steganography techniques[J]. IEEE Access, 2021, 9: 31805-31815.
- [12] LEE J, LEE H. An ssh predictive model using machine learning with web proxy session logs [J]. International Journal of Information Security, 2022, 21(2).
- [13] 周昌海, 樊玲, 谷云琦. 基于 C/S 架构 & 对称加密的图书馆有毒有害读物自动筛查及下架

系统软件设计[J]. 信息与电脑 (理论版), 2022, 34(24): 127-130.

- [14] 余晓青. 新时代意识形态网络舆情治理研究[D]. 福州: 福建师范大学, 2019.
- [15] 蔡克难. 多方面努力保证出版物质量[J]. 编辑学刊, 1994(4): 19-21.
- [16] 陈朝荣. 数据库的一种动态加密法[J]. 电脑编程技巧与维护, 1995(2): 57-58.
- [17] 刘艳. 基于 C/S 模式的远程网络视频监控系统的设计和实现[J]. 新型工业化, 2022, 12(09): 264-267.
- [18] 教文兵. 基于 H.264/AVC 屏幕录制回放系统[D]. 华中科技大学, 2012.
- [19] MARPE D, WIEGAND T, SULLIVAN G J. The h.264/mpeg4 advanced video coding standard and its applications[J]. IEEE Communications Magazine, 2006, 44(8): 134-143.
- [20] 裴世保, 李厚强, 俞能海. H.264/AVC 帧内预测模式选择算法研究[J]. 计算机应用, 2005, 25(8): 71-73.
- [21] 宋彬, 常义林, 李春林. H.264 帧间预测模式的快速选择算法[J]. 电子学报, 2007(4): 697-700.
- [22] HUANG Y W, HSIEH B Y, CHIEN S Y, et al. Analysis and complexity reduction of multiple reference frames motion estimation in h.264/avc[J]. IEEE Transactions on Circuits and Systems for Video Technology, 2006, 16(4): 507-522.
- [23] SUH J, JEONG J. Fast sub-pixel motion estimation techniques having lower computational complexity[J]. IEEE Transactions on Consumer Electronics, 2004, 50(3): 968-973.
- [24] MALVAR H, HALLAPURO A, KARCZEWICZ M, et al. Low-complexity transform and quantization in h.264/avc[J]. IEEE Transactions on Circuits and Systems for Video Technology, 2003, 13(7): 598-603.

致谢

本文能顺利完成，首先最要感谢的是导师与学校期刊资源的支持。

为了能让监考工作更顺利更简便的进行，才以此为题开发了这样一个系统，在这个系统的帮助下，监考与参考人员只需要将精力放在参加考试与监管的工作上，其余的工作都由本系统完成。

在开发这个系统的过程中，遇到了很多困难，同时也学会了很多专业知识，在此感谢学校与导师的帮助。